

Toward Robust End-to-End Delay Prediction: A GNN Approach With Routing-Aware Attention and Masked Subgraph Sampling

Zichen Wang¹, Yiqi Chen¹, Dongwei Liu, Huile Wang¹, Yusong Zhou¹, and Weiping Zheng¹

Abstract—The end-to-end delay prediction is critical for intelligent network management, particularly in latency-sensitive and dynamic environments. While recent deep learning (DL) models have shown promising results, their reliance on sequential encoding of routing paths limits generalization to unseen routing schemes. In this article, we propose a robust graph neural network (GNN)-based delay prediction model that overcomes this limitation by introducing a global routing representation and a routing-aware attention mechanism. The model queries flow-relevant features from a unified topology-routing map without relying on routing sequences. In addition, a mask-based subgraph sampling strategy enables the model to infer global routing correlations from partial flow interactions, further enhancing its adaptability. Extensive experiments are conducted on four public datasets—TnCwD, NSFNET, GBN, and GEANT2. The results demonstrate that our model not only outperforms existing methods in prediction accuracy but also exhibits strong generalization across diverse routing configurations. Future work will focus on further validating the model under more complex and dynamic routing scenarios.

Index Terms—Attention mechanism, end-to-end delay prediction, flow interaction graph (FIG), graph neural network (GNN), routing generalization.

I. INTRODUCTION

WITH the widespread adoption of cloud computing, the Internet of Things (IoT) [1], [2], and the Industrial Internet, modern network infrastructures have grown increasingly complex, imposing greater demands on real-time monitoring and intelligent management. Among various performance metrics, end-to-end delay—the total time for a data packet to travel from source to destination—has become a critical indicator. It is essential not only for ensuring quality of service (QoS) in latency-sensitive applications such as voice communication, video conferencing, and AR/VR, but also plays a key role in resource scheduling [3], congestion control [4], and the operation of network digital twins (NDTs) [5], [6]. Precise prediction of end-to-end delay enables effective

routing optimization [7], [8], congestion mitigation, and the accurate modeling of network behavior, establishing it as an indispensable component of intelligent and adaptive network management frameworks.

Various methods have been proposed for end-to-end delay prediction, ranging from traditional approaches to machine learning and deep learning (DL)-based models. Traditional methods can be broadly categorized into two types: analytical models (e.g., queuing theory [9], [10] and network calculus [11], [12], [13]) and simulator-based approaches (e.g., NS3 [14] and OMNet++ [15]). Analytical models are often inadequate for modeling real-world networks due to their reliance on strong and often unrealistic assumptions. While packet-level network simulators offer high-fidelity emulation, their computational cost increases linearly with the volume of forwarded packets, limiting scalability in large-scale scenarios. To address these limitations, machine learning-based models—such as causal Bayesian networks (CBNs) [16] and support vector regression (SVR) [17]—have been introduced for network delay prediction and have demonstrated promising results under relatively simple network conditions. Recently, DL has become the dominant approach for end-to-end delay prediction due to its strong performance and capacity to model complex and nonlinear network dynamics.

To accurately model end-to-end delay, DL models can leverage various inputs, such as traffic matrices, network topology, and routing schemes. Mestres et al. [18] were the first to apply DL for delay prediction, modeling delay as a function of traffic matrices. Krasniqi et al. [19] used a multilayer perceptron (MLP) with flow-level traffic and link capacity as inputs. While simple, these methods suffer from limited accuracy due to the omission of key contextual information, such as topology and routing. To overcome this limitation, recent studies have proposed graph neural network (GNN)-based models—such as RouteNet [20] and FI-Graphormer [21]—which explicitly incorporate topological and routing information, resulting in significantly improved performance. For example, RouteNet simulates packet propagation by iteratively updating link and path states through a message-passing mechanism, utilizing traffic, topology, and routing information as input. FI-Graphormer constructs a flow interaction graph (FIG) to capture resource contention among flows, encodes both traffic and routing information onto this graph, and employs a graph Transformer to predict end-to-end delay.

Received 25 June 2025; revised 8 November 2025; accepted 24 February 2026. Date of publication 26 March 2026; date of current version 2 September 2026. This work was supported by the National Key Research and Development Program of China under Grant 2019YFB1804003. (Corresponding author: Weiping Zheng.)

The authors are with the School of Computer Science, South China Normal University, Guangzhou 510631, China (e-mail: zichenwang@m.scnu.edu.cn; yiqichen@m.scnu.edu.cn; liudongwei@m.scnu.edu.cn; huilewang@m.scnu.edu.cn; yusongzhou@m.scnu.edu.cn; zhengweiping@scnu.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2026.3670186

2162-237X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09, 2026 at 12:29:31 UTC from IEEE Xplore. Restrictions apply.

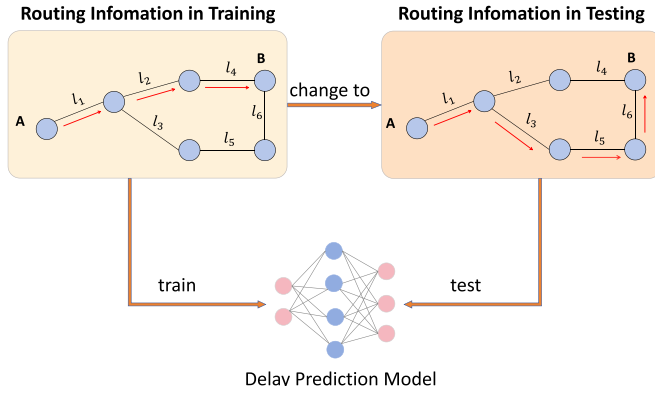


Fig. 1. Routing information is utilized to improve end-to-end delay prediction. However, the model may suffer from routing generalization issues, where unseen routing schemes lead to prediction degradation.

Despite their strong predictive performance, these methods face a key challenge: *limited generalization to unseen routing schemes*. As shown in Fig. 1, when trained on one dataset and evaluated on another with different routing configurations, their prediction accuracy degrades significantly. This limitation stems from the *sequential encoding* mechanisms employed by both models. In RouteNet, path states are updated using a recurrent neural network (RNN) that encodes link states in the order of packet transmission. Similarly, FI-Graphormer encodes flow features through a routing-guided sequence of link attributes. Thus, a critical challenge remains: how to effectively leverage network topology and routing information to enhance prediction accuracy while improving the generalization to dynamic routing scenarios.

In this article, we propose a novel GNN-based end-to-end delay prediction model that exhibits both superior predictive performance and strong generalization capability. The core idea is to develop a routing knowledge representation that eliminates the reliance on sequential encoding, thereby enhancing generalization across diverse routing schemes. Following FI-Graphormer, we also utilize an FIG to capture interactions among flows. However, instead of relying on sequential encoding, we first construct a global routing map that encodes both topological and routing information and query the corresponding features for each FIG node from the embedding of this unified map. To effectively encode the topological and routing information, we transform the original network topology into a line graph and propose a node encoding method to integrate routing information into this structure (i.e., the map). To obtain unique features for each FIG node, we introduce a routing-aware attention mechanism, which retrieves the most relevant part of the map embedding based on the flow feature. Traffic volume is included in the flow feature; however, as multiple flows may share the same traffic value, we incorporate a learnable flow ID embedding to improve feature distinctiveness. To ensure that the map embedding effectively captures routing and topological semantics, we further propose a mask-based random sampling strategy to generate FIG subgraphs, enabling the model to infer global routing correlations from partial

transmission views. This enhances the model's ability to adapt to routing dynamics in real-world networks.

Our main contributions are summarized as follows.

- 1) We propose a novel GNN-based model for end-to-end delay prediction that achieves both high prediction accuracy and strong generalization capability across diverse routing configurations. This addresses the critical limitation of existing methods that fail to generalize under unseen routing schemes.
- 2) We design a routing-aware attention mechanism and a mask-based subgraph sampling strategy, which together allow the model to retrieve relevant routing features without relying on sequential encoding and infer global routing correlations from incomplete flow interactions, improving adaptability to real-world routing dynamics.
- 3) Our proposed model demonstrates superior performance compared with existing state-of-the-art methods, as validated by experimental results on four public datasets—TnCwD, NSFNET, GBN, and GEANT2. In addition, our model also shows strong generalization capability to unseen routing schemes.

The rest of this article is organized as follows. In Section II, a review of related studies is presented. Section III describes the problem and provides detailed information on the proposed method. Section IV includes the experimental results. Finally, the conclusions are presented in Section V.

II. RELATED WORKS

The network performance modeling plays a critical role in network optimization by providing accurate and reliable QoS metrics predictions. Recently, DL methods have become dominant in network performance modeling due to their remarkable results.

A. DL-Based Network Performance Modeling

DL offers a set of techniques [22], [23], [24] for building accurate data-driven models of network performance, allowing the model to be trained directly on real data without relying on assumptions about the underlying physical network. For example, xWeaver [25] is a traffic-driven DL solution to explore the optimal network topology given a traffic scheme. DeepComNet [26] uses gated GNNs (GGNNs) [27] to perform QoS metrics prediction. Li et al. [28] proposed a GNN model that can generalize to previously unseen networks and provide accurate FCT estimates. Mestres et al. [18] treated the network as a closed box that maps traffic matrices to mean end-to-end delay, training various neural network models and evaluating their performance across different topologies, traffic intensities, and routing schemes. Xiao et al. [29] designed a variational autoencoder (VAE) model called deep-Q, enhanced with long short-term memory (LSTM) networks [30], which can infer various QoS metrics on different networks based on given real-time traffic conditions. Suzuki et al. [31] reformulated the delay prediction as a classification task, utilizing a graph convolutional network (GCN) with a log-softmax classifier in a semi-supervised setting.

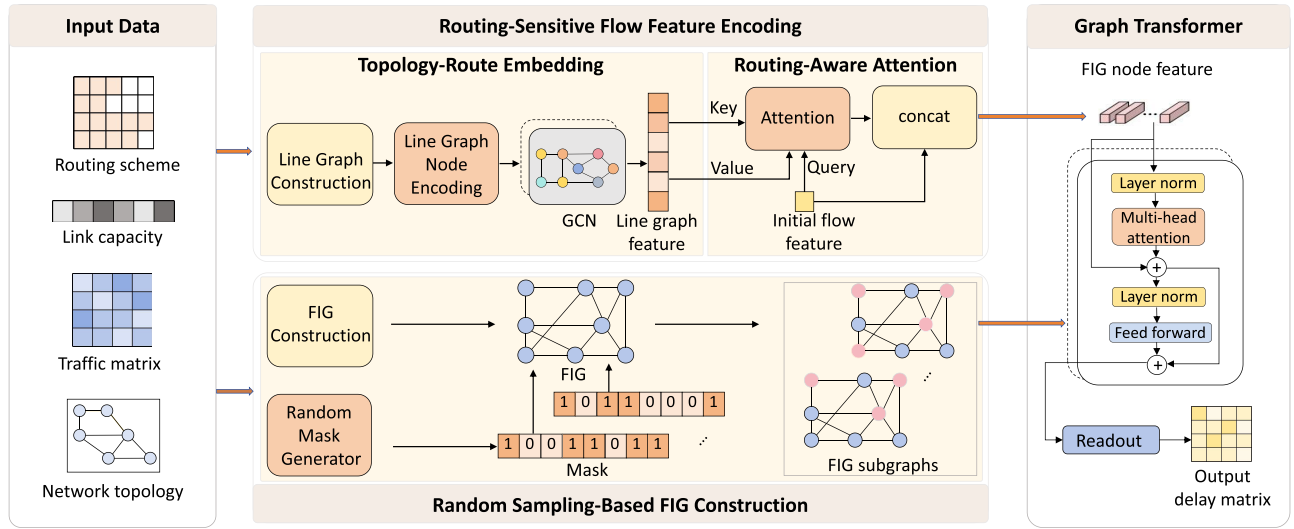


Fig. 2. Architecture of our proposed model consists of three components: the routing-sensitive flow feature encoding block, the random sampling-based FIG construction block, and the graph Transformer block. The first block prepares routing-sensitive features for each FIG node using a routing-aware attention mechanism. The second block constructs FIG subgraphs using a random mask generator. Based on the generated subgraphs and node features, the graph Transformer updates the features and ultimately produces the predicted end-to-end delays.

B. GNN-Based End-to-End Delay Prediction

RouteNet [20] is a novel GNN-based end-to-end delay prediction model that captures the complex relationships among network topology, routing schemes, and traffic matrices and generalizes effectively to unseen topological configurations. Its core principle lies in the iterative update mechanism between paths and links, which enables the model to explicitly learn routing behavior within the target network. Building upon RouteNet, several follow-up studies have emerged, including RouteNet-Erlang [32] and RouteNet-Fermi [33]. Compared with RouteNet's serialized update mechanism, PLNet [34] reconstructs the network representation using a bipartite graph structure that decouples path nodes from link nodes. It adopts a parallel message-passing architecture that mitigates the impact of link ordering within paths and improves computational efficiency, particularly as path lengths increase. Similar to RouteNet, TransNet [35] predicts the end-to-end delay in 6G through iterative updates of link and path states; however, it uses a Transformer-based architecture in their updates. Zheng et al. [21] proposed a graph Transformer-based model for end-to-end delay prediction, where an FIG is utilized to explicitly model resource contention between flows. Unlike previous studies, xNet [36] offers a general approach to modeling network features by leveraging relational graph representations and configurable GNN blocks. These components can be configured and instantiated to model network behavior in different scenarios. Existing work often does not fully consider the generalization capability of models under different routing schemes. This article proposes a novel method to cope with dynamically changing routing scenarios in networks.

III. METHODOLOGY

A. Problem Description

Given a network topology TP with N nodes and L edges, where there are $N(N-1)$ origin-destination (OD) flows, a routing scheme R that describes the link-sequence information for

each OD flow, a set $C = \{c_1, c_2, \dots, c_L\}$ presenting the capacity of each link in TP , and a traffic matrix $TM \in \mathbb{R}^{N \times (N-1)}$ that specifies the traffic volume for each OD pair, our goal is to infer the end-to-end delay matrix $DM \in \mathbb{R}^{N \times (N-1)}$ for all OD flows. The end-to-end delay prediction problem can be formulated as follows:

$$DM = \mathcal{F}(TP, R, C, TM). \quad (1)$$

B. Overall Architecture

To handle routing dynamics, we propose a graph Transformer-based end-to-end delay prediction model with strong generalization ability across changing routing patterns. As demonstrated in Fig. 2, the overall architecture of the model contains three parts: 1) random sampling-based FIG construction; 2) routing-sensitive flow feature encoding; and 3) graph Transformer.

In the random sampling-based FIG construction module, we first construct an FIG to explicitly capture the resource competition among OD flows. To improve the generalization capability of the model, we adopt a random sampling strategy to generate FIG subgraphs, encouraging the model to infer the global routing information from local routing patterns. In the routing-sensitive flow feature encoding module, we utilize a GCN [37] as a graph feature extractor to generate line graph features that encode both topological and routing information. Subsequently, a routing-aware attention mechanism is applied to obtain routing-sensitive flow representations for each node in the FIG. Finally, a graph Transformer is used to update node representations within each sampled FIG subgraph, and a readout function is applied to compute the final end-to-end delay matrix.

C. Random Sampling-Based FIG Construction

1) *FIG Construction*: Competition for shared link resources is indeed a crucial factor influencing end-to-end

delay. Consequently, we follow the FI-Graphormer [21] to explicitly model the resource competitions by constructing the FIG. Formally, we represent the FIG as $G = (V, E)$, where V denotes the node set, with each node being an OD flow. E denotes the edge set, where an edge exists between two nodes if their corresponding OD flows share at least one common link.

2) *Random Sampling Strategy*: In this article, we adopt a mask-based random sampling strategy to generate multiple FIG subgraphs from a single FIG, aiming to enhance the diversity of training samples. This strategy is motivated by two main considerations. First, training on multiple subgraph samples encourages the model to learn from diverse local perspectives, thereby enabling it to infer global correlations between end-to-end delay and flow interactions, and ultimately improving its generalization capacity to varying routing dynamics. Second, the use of masked subgraphs helps mitigate overfitting, leading to better adaptability under dynamic routing configurations.

3) *FIG Subgraph Generation*: To construct a subgraph from the FIG, we first apply a random mask generator using a binary masking mechanism. Specifically, a node-level binary mask $M \in \{0, 1\}^{N(N-1) \times 1}$ is sampled, where each element $M_i = 1$ indicates that the i th flow node is retained (i.e., sampled), while $M_i = 0$ indicates that the node is excluded. To ensure the sufficient structural and semantic information is preserved, the sampling rate is maintained above a certain threshold (e.g., $\geq 60\%$). Applying the mask M to the original FIG yields a subgraph by removing the unselected nodes and their associated edges. This procedure can be repeated multiple times on the same FIG to generate diverse subgraph instances for training.

D. Routing-Sensitive Flow Feature Encoding

To construct order-independent flow representations, we first build a global routing information map, from which each flow can access routing features relevant to itself. This enables the extraction of routing representations that are independent of the link order. Therefore, we introduce two modules to construct the routing map and to query the relevant flow features, respectively.

1) *Topology-Route Embedding*: To produce the aforementioned map, we encode the topological and routing information into a unified embedding. To achieve this integration, we propose a novel topology-route embedding method, in which a line graph is used to represent the topological structure, while the routing information is encoded as node features associated with the line graph.

Line Graph Construction: Given a network topology TP , we construct its corresponding line graph I_{TP} by mapping each edge in TP to a node in I_{TP} . An edge is added between two nodes in I_{TP} if their corresponding edges in TP share a common endpoint.

Line Graph Node Encoding: The encoding information includes the intrinsic attributes of each link in TP as well as the routing information related to the link. The intrinsic link attributes include link capacity c and link usage u . Since each flow may traverse several links, we define a set $S_{l_i} = \{f \mid l_i \in f, f \in F\}$, where F denotes the entire set of OD

flows, and S_{l_i} collects all flows that traverse link l_i . The link usage is obtained by summing the traffic values of all flows traversing a given link, computed using the following formula:

$$u_i = \sum_{j \in S_{l_i}} tm_j \quad (2)$$

where tm_j is the traffic volume in TM . To incorporate routing-related information, we first update the set of flows traversing link l_i using the previously sampled FIG subgraph. Specifically, let $S'_{l_i} = \{f \mid l_i \in f, f \in F'\}$, where F' is the set of OD flows retained after random sampling. Then, only the flows in F' are used to encode the nodes of the line graph, ensuring that the line graph node encoding contains only the locally sampled routing information. The complete node encoding for l_i is then given by

$$Y_{l_i} = [c_{l_i}, u_{l_i}, tm_{f_i(0)}, tm_{f_i(1)}, \dots, tm_{f_i(T-1)}] \quad (3)$$

where $tm_{f_i(k)}$ represents the traffic value in TM of the k th flow in S'_{l_i} . As the length of S'_{l_i} varies across different links, we enforce a fixed encoding length of T flows traversing per link. Concretely, if $|S'_{l_i}| > T$, we truncate the list to the first T flows; if $|S'_{l_i}| < T$, we pad with zeros.

Line Graph Feature Extraction: We employ a two-layer GCN as an extractor to derive a graph-level representation. Let $Y \in \mathbb{R}^{L \times d_l}$ be the original node feature matrix, where d_l is the dimension of the node feature, and $A \in \{0, 1\}^{L \times L}$ denote the adjacency matrix of I_{TP} . By adding self-loops, the adjacency matrix becomes $\hat{A} = A + I$, and I is the identity matrix. The first GCN layer performs spectral graph convolution to propagate and transform node features as follows:

$$H^{(1)} = \text{LeakyReLU}(\tilde{A}YW^{(0)}) \quad (4)$$

where $\tilde{A} = \hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}$ denotes the symmetrically normalized adjacency matrix. $W^{(0)} \in \mathbb{R}^{d_l \times d_h}$ is a trainable weight matrix with hidden dimension d_h , and $\hat{D}_{ii} = \sum_j \hat{A}_{ij}$. The second GCN layer then further aggregates information from higher order neighborhoods as follows:

$$H^{(2)} = \tilde{A}H^{(1)}W^{(1)} \quad (5)$$

where $W^{(1)} \in \mathbb{R}^{d_h \times o}$, o indicates the output dimension. Subsequently, the node-level feature matrix $H^{(2)} \in \mathbb{R}^{L \times o}$ is aggregated into a single graph-level feature g using global mean pooling

$$g = \frac{1}{L} \sum_{i=1}^L H_i^{(2)}. \quad (6)$$

As a result, the graph-level feature $g \in \mathbb{R}^{1 \times o}$ serves as a routing map that contains the routing information along with the network topology. OD flows can then extract their routing details from g .

2) *Routing-Aware Attention*: In this article, we propose a routing-aware attention method to fetch routing-related features from the routing map g for each FIG node.

To query the map, we first assign each FIG node an initial feature comprising its traffic value and a unique flow ID, which distinguishes OD flows that may share the same traffic matrix value. A learnable embedding layer is employed to generate the flow ID embedding $e_i \in \mathbb{R}^{1 \times D}$, where D denotes the

embedding dimension. The initial encoding x_i of a flow node v_i is then defined as follows:

$$x_i = [e_i, tm_i] \quad (7)$$

where $x_i \in \mathbb{R}^{1 \times D'}$. To ensure dimensional alignment with g , x_i is projected into $q_i \in \mathbb{R}^{1 \times o}$ through a linear transformation.

Multihead attention is then applied as follows:

$$Q = qW_Q, K = gW_K, V = gW_V \quad (8)$$

where $W_Q \in \mathbb{R}^{o \times d_K}$, $W_K \in \mathbb{R}^{o \times d_K}$, and $W_V \in \mathbb{R}^{o \times d_V}$ are learnable projection matrices. We set $d_K = d_V = o/n$, where n is the number of attention heads. The output of the single-head attention is computed as follows:

$$r = \text{softmax}\left(\frac{QK^T}{\sqrt{d_K}}\right)V. \quad (9)$$

The outputs from n heads are concatenated to form $\hat{r}$, which is then linearly projected to obtain the final feature $\tilde{r}$.

This feature is concatenated with the initial flow feature to form the complete flow feature $\hat{q}$

$$\hat{q} = \text{concat}(q, \tilde{r}) \quad (10)$$

where $\hat{q} \in \mathbb{R}^{N(N-1) \times d_f}$.

E. Graph Transformer

To capture spatial dependencies among flows, we apply a graph Transformer to update the FIG node features obtained from the previous module on the sampled FIG subgraph.

Graph Transformer Encoder: The proposed graph Transformer consists of K encoder layers, where K is chosen empirically based on validation performance. Within the l th layer, the query $Q^{(l)}$, key $K^{(l)}$, and value $V^{(l)}$ matrices are obtained through three independent linear projections

$$Q^{(l)} = h^{(l)}W_Q^{(l)}, K^{(l)} = h^{(l)}W_K^{(l)}, V^{(l)} = h^{(l)}W_V^{(l)} \quad (11)$$

where $d_K = d_V = d_f/m$, $W_Q^{(l)} \in \mathbb{R}^{d_f \times d_K}$, $W_K^{(l)} \in \mathbb{R}^{d_f \times d_K}$, $W_V^{(l)} \in \mathbb{R}^{d_f \times d_V}$, $h^{(0)} = \hat{q}$, and m denote the number of attention heads. The attention scores are then computed as follows:

$$a^{(l)} = \text{softmax}\left(\frac{Q^{(l)}(K^{(l)})^T}{\sqrt{d_K}}\right). \quad (12)$$

Given the attention scores $a^{(l)}$, the output of a single attention head at layer l is computed as follows:

$$z^{(l)} = a^{(l)}V^{(l)}. \quad (13)$$

The outputs of m attention heads are concatenated to form $\hat{z}^{(l)}$. Using $\hat{z}^{(l)}$ as input, a feedforward network layer with a residual connection is applied, obtaining the output of the l th Transformer layer

$$h^{(l+1)} = \text{FFN}(\hat{z}^{(l)}). \quad (14)$$

Readout Function: Finally, a readout function is defined to predict the end-to-end delay matrix DM . The output flow node features from the final Transformer layer, denoted by $h^{(K)}$,

are used as input, and a fully connected layer is employed to perform the readout, as follows:

$$DM = h^{(K)}W_r \quad (15)$$

where $W_r \in \mathbb{R}^{d_f}$, $DM \in \mathbb{R}^{N(N-1)}$ denotes the predicted end-to-end delay matrix.

IV. EXPERIMENTS

A. Experimental Settings

1) *Datasets:* We use four publicly available datasets for evaluation: TnCwD, NSFNET, GBN, and GEANT2.

TnCwD [19]: The dataset contains 1 750 000 samples, generated by simulating the Abilene network using the NS 3 simulator. The Abilene topology consists of 12 nodes, 15 links, and 132 paths. During simulation, various link capacities, propagation delays, and traffic intensities are configured under a shortest path first routing strategy. Since TnCwD reflects realistic traffic conditions, it presents greater challenges for accurate prediction. The dataset includes both traffic matrices and end-to-end delays. In accordance with [19], we randomly split the dataset into training, validation, and test sets following a ratio of 16:4:5.

NSFNET [20]: The dataset consists of 125 000 samples, generated by simulating the NSF network [38] using the OMNeT++ simulator. The NSF topology comprises 14 nodes, 42 links, and 182 paths. During simulation, multiple routing schemes—including shortest path first and max-min routing—are employed to emulate diverse network conditions. The dataset provides both traffic matrices and the corresponding average end-to-end delays. We randomly partition these samples into training, validation, and test sets with proportions of 28:7:15, respectively.

GBN [20]: The dataset also comprises 125 000 samples, obtained from simulations on the GBN [39] topology, which consists of 17 nodes, 52 links, and 272 paths. The same data partitioning is applied as that used for the NSFNET dataset.

GEANT2 [20]: This dataset also includes 125 000 samples, obtained from simulations on the GEANT topology [40], consisting of 24 nodes, 74 links, and 552 paths. The same data partitioning strategy as in the NSFNET dataset is applied.

2) *Evaluation Metrics:* To evaluate the performance of our model, we use four metrics: root-mean-squared error (RMSE), the coefficient of determination (R^2), Pearson correlation coefficient (PCC), and mean absolute error (MAE). Their definitions are given as follows:

$$RMSE = \sqrt{\frac{1}{s} \sum_{i=1}^s (X_i - Y_i)^2} \quad (16)$$

$$R^2 = 1 - \frac{\sum_{i=1}^s (X_i - Y_i)^2}{\sum_{i=1}^s (\bar{Y} - Y_i)^2} \quad (17)$$

$$PCC = \frac{\sum_{i=1}^s (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^s (\bar{X} - X_i)^2} \sqrt{\sum_{i=1}^s (\bar{Y} - Y_i)^2}} \quad (18)$$

$$MAE = \frac{1}{s} \sum_{i=1}^s |X_i - Y_i| \quad (19)$$

where X_i is the i th predicted end-to-end delay value; Y_i is the corresponding ground truth, $\bar{X}$ and $\bar{Y}$ are the corresponding mean values, and s is the sample size.

3) *Baselines*: We compare our model with nine baseline models, including current state-of-the-art methods, popular GNN models, and spatiotemporal prediction models, as described below.

FI-Graphormer (2024) [21]: FI-Graphormer explicitly captures interflow resource competition for the first time by constructing an FIG. It incorporates link information into each flow node's encoding, thereby modeling both competitions among flows and dependencies between flows and links.

RouteNet (2020) [20]: RouteNet is a novel network model based on GNN. It employs message-passing neural networks to achieve accurate end-to-end network delay prediction.

PLNet (2021) [34]: PLNet adopts a bipartite graph structure to reconstruct the network representation, decoupling path-link connections. Compared with RouteNet's serialized update mechanism, PLNet employs a parallelized message-passing architecture that eliminates the influence of link order within each path.

NN (2020) [19]: NN employs a multilayer perceptron (MLP) [41] to predict end-to-end delay using the traffic volume and link capacity information. However, this model does not take network topology or routing information into account.

GAT: We apply the GAT method [42] on the FIG, remaining the other part of FI-Graphormer unchanged.

FCSGNN (2024) [43]: FCSGNN employs a fully connected (FC) graph construction module and FC graph convolution module to learn spatiotemporal dependencies in multivariate time-series data.

DGCRN (2023) [44]: DGCRN is a spatiotemporal model that integrates GNNs with gated recurrent units (GRUs) [45] to capture spatial dependencies in traffic networks that evolve over time.

STPGNN (2024) [46]: STPGNN identifies critical nodes for traffic aggregation and applies graph convolutions to both the key-node subgraph and the full network. A weighted fusion of both views captures spatiotemporal dependencies at local and global levels.

TimeMixer (2024) [47]: TimeMixer is an MLP-based architecture that uses PDM and FMM blocks to exploit disentangled multiscale series for past extraction and future prediction.

4) *Implementation Details*: The proposed model and baselines are implemented on a CentOS 7.9-2009 workstation using PyTorch 2.3.1, Python 3.8.18, and CUDA 11.8, with two NVIDIA A100 GPUs with 80-GB VRAM. For our model, an Adam optimizer is applied. Mean squared error (MSE) is employed as the loss function. The initial learning rate is set to 0.001 for the TnCwD dataset and 0.0001 for the NSFNET, GBN, and GEANT2 datasets. The learning rate decays by a factor of 0.5 every 40 epochs. The total number of epochs is 100. The batch size is 16, and the dropout ratio is 0.5. The Transformer encoder layer of our model is 14, and the number of attention heads is 4. The attention layer in the routing-aware attention module is 1, and the number of attention heads is also 4. The RouteNet model uses the

parameter settings from [20]. The NN model adopts the same hyperparameter configuration as in [19]. For the FCSGNN, DGCRN, STPGNN, and TimeMixer models, we have trained each model with different time step settings (5, 10, and 15) and selected the configuration with the best performance at $5 \times$ steps as the final result.

B. Comparison With State-of-the-Art Methods

1) *Performance Comparison With Complete Data*: The performance results of our proposed model and nine baselines on the four datasets are presented in Table I. It is shown that our model outperforms the baselines on all four datasets, according to most evaluation metrics. This demonstrates that our model's predictive performance is significantly better than baselines. Among the four datasets, the TnCwD dataset presents the greatest challenge for end-to-end delay prediction, due to its more complex simulation of mixed TCP and UDP traffic. On the TnCwD dataset, RMSE, R^2 , PCC, and MAE values of our model are 0.0206, 0.9314, 0.9662, and 0.0103, respectively; compared with the best baseline model (i.e., the FI-Graphormer), these are improvements of 9.25%, 1.56%, 0.64%, and 12.71%, respectively. On the NSFNET and GBN datasets, our model also outperforms all baselines. Among these baselines, PLNet achieves the best values on the NSFNET dataset, and FI-Graphormer achieves the best values on the GBN dataset. Specifically, RMSE, R^2 , PCC, and MAE values of our model are 0.2001 (0.1496), 0.9584 (0.9876), 0.9806 (0.9945), and 0.0921 (0.0773), respectively, on NSFNET (GBN); compared with the best baseline models, it achieves improvements of 1.48% (39.08%), 0.13% (2.15%), 0.22% (0.40%), and -6.22% (33.42%), respectively. However, on the NSFNET dataset, our model's MAE (0.0921) is slightly higher than that of PLNet. The experimental results on the large-scale GEANT2 dataset further demonstrate the competitive performance of our model, achieving RMSE, R^2 , PCC, and MAE values of 0.1339, 0.9857, 0.9947, and 0.0637, respectively. Notably, our model ranks first on R^2 and RMSE, and second on PCC and MAE, confirming its strong overall effectiveness.

FI-Graphormer obtains competitive performance across all four datasets, demonstrating the effectiveness of capturing interflow resource competition and flow-link dependencies. Both RouteNet and PLNet achieve competitive results on the NSFNET, GBN, and GEANT2 datasets, but perform weaker on the TnCwD dataset. The reason may be that TnCwD simulates a more complex traffic scenario. NN achieves good results on the TnCwD dataset but performs very poorly on the NSFNET, GBN, and GEANT2 datasets. This discrepancy can be attributed in part to suboptimal hyperparameter choices and, more importantly, to the NN's inability to explicitly leverage network topology information. GAT performs worst on the four datasets, demonstrating the effectiveness of graph Transformer. FCSGNN, DGCRN, STPGNN, and TimeMixer all require sequential data and can thus only be evaluated on the TnCwD dataset, where their performance is comparable with NN.

2) *Performance Comparison With Random Missing*: To evaluate the model's robustness against uncorrelated

TABLE I

COMPARISON RESULTS BETWEEN OUR MODEL AND BASELINES ($\times 10^{-2}$). THE BEST RESULTS ARE IN BOLD. THE SECOND-BEST RESULTS ARE INDICATED WITH UNDERLINE

Dataset	Metric	Model									
		Ours	FI-Graphormer	RouteNet	PLNet	NN	GAT	FCSGNN	DGCRN	STPGNN	TimeMixer
TnCwD	RMSE	2.06	<u>2.27</u>	3.14	3.29	3.52	4.38	3.63	3.54	3.69	4.05
	R ²	93.14	<u>91.71</u>	84.14	82.58	80.06	69.13	78.74	79.81	78.00	73.52
	PCC	96.62	<u>96.01</u>	91.80	90.90	89.59	85.44	88.84	89.35	88.32	87.55
	MAE	1.03	<u>1.18</u>	2.38	2.51	2.76	3.40	2.88	2.67	2.85	3.21
NSFNET	RMSE	20.01	25.62	28.95	<u>20.31</u>	69.80	75.46	—	—	—	—
	R ²	95.84	93.63	91.30	<u>95.72</u>	49.47	40.96	—	—	—	—
	PCC	98.06	97.21	95.56	<u>97.84</u>	76.95	64.05	—	—	—	—
	MAE	9.21	7.85	14.73	<u>8.67</u>	38.39	52.21	—	—	—	—
GBN	RMSE	14.96	<u>24.56</u>	37.97	32.08	83.07	89.58	—	—	—	—
	R ²	98.76	<u>96.68</u>	92.07	94.34	62.09	55.95	—	—	—	—
	PCC	99.45	<u>99.05</u>	95.96	97.19	81.06	74.82	—	—	—	—
	MAE	7.73	<u>11.61</u>	21.44	15.03	45.99	63.48	—	—	—	—
GEANT2	RMSE	13.39	<u>13.73</u>	40.04	31.89	95.19	82.47	—	—	—	—
	R ²	98.57	<u>98.49</u>	87.24	91.90	27.95	45.89	—	—	—	—
	PCC	<u>99.47</u>	99.70	93.40	95.86	59.75	68.27	—	—	—	—
	MAE	<u>6.37</u>	5.84	22.84	14.45	48.28	52.97	—	—	—	—

TABLE II

COMPARISON RESULTS BETWEEN OUR MODEL AND BASELINES IN TRAFFIC FAILURE CONDITIONS ($\times 10^{-2}$). THE BEST RESULTS ARE IN BOLD. THE SECOND-BEST RESULTS ARE INDICATED WITH UNDERLINING

Dataset	Metric	Model									
		Ours	FI-Graphormer	RouteNet	PLNet	NN	GAT	FCSGNN	DGCRN	STPGNN	TimeMixer
RandomMissing-5%	RMSE	2.32	3.42	<u>3.19</u>	3.56	3.51	4.66	3.66	3.58	3.75	5.92
	R ²	91.37	81.18	<u>83.66</u>	79.60	80.10	65.08	78.51	79.43	77.36	43.61
	PCC	95.92	90.89	<u>91.50</u>	89.48	89.61	82.85	88.70	89.14	88.00	73.68
	MAE	1.20	<u>2.08</u>	2.42	2.74	2.76	3.59	2.90	2.69	2.89	4.72
RandomMissing-10%	RMSE	2.77	4.71	<u>3.36</u>	3.66	3.52	5.11	3.80	3.62	3.95	8.91
	R ²	87.66	64.35	<u>81.82</u>	78.45	80.09	58.02	76.74	78.90	74.94	−27.49
	PCC	94.23	82.59	<u>90.70</u>	88.95	89.61	78.07	87.75	88.85	87.04	57.48
	MAE	1.60	3.14	<u>2.54</u>	2.80	2.76	3.91	3.00	2.71	3.05	7.10
Structured-Missing	RMSE	2.81	4.33	<u>3.50</u>	3.91	3.52	4.68	3.82	3.68	3.92	9.49
	R ²	87.35	69.89	<u>80.30</u>	75.37	80.11	64.82	76.52	78.20	75.27	−44.71
	PCC	94.19	84.87	<u>89.75</u>	87.50	89.62	81.87	87.87	88.49	86.97	58.37
	MAE	1.56	2.85	<u>2.63</u>	2.97	2.76	3.62	3.00	2.74	3.00	7.47

and distributed traffic failures, we construct two test sets, RandomMissing-5% and RandomMissing-10%, based on the original TnCwD test set by randomly removing 5% and 10% of the flow data, respectively. To maintain a consistent input structure, all missing flow information was imputed with the mean values of the remaining available flows. This approach simulates scenarios in which local failures compromise the overall integrity of network information.

We train our model on the TnCwD training set and evaluate it on the RandomMissing-5% and RandomMissing-10% testing sets. The results, summarized in Table II, show that as the missing rate increases, the performance of all models degrades to some extent. Nevertheless, under both missing conditions, our proposed model consistently outperforms all

baselines, and the performance gap remains substantial. These results demonstrate that our model preserves strong inherent robustness and generalization capability even when part of the input traffic data is missing. We attribute this advantage to the masked subgraph sampling mechanism. Among the baseline methods, FI-Graphormer and TimeMixer experience the most significant performance decline, highlighting their limitations in real-world scenarios.

3) *Performance Comparison With Structured Missing:* We further employ a structured-missing strategy to simulate a more realistic traffic failure scenario. Based on the TnCwD test set, we construct the Structured-Missing test set. Specifically, for each sample, we randomly select one link and remove all flows traversing the link. The links are selected randomly, and

TABLE III

COMPARISON OF THE NUMBER OF PARAMETERS, FLOPs, TRAINING TIME, TEST TIME (EVALUATED ON THE TnCwD DATASET), AND GPU MEMORY FOR VARIOUS METHODS

	Ours	FI-Graphormer	RouteNet	PLNet	NN	GAT	FCSGNN	DGCRN	STPGNN	TimeMixer
Model Params (M)	20.659	18.414	0.0872	0.3631	1.519	0.0003	0.0465	0.1917	0.3000	0.2165
FLOPs (G)	38.994	38.826	45.19	0.646	0.024	0.061	15.342	5.076	6.826	2.364
GPU Memory (MiB)	11086	10974	9923	10366	9929	9916	12515	10682	10937	11137
Training Time (s/epoch)	331	434	588	531	13	449	34	32	11	30
Test Time (s)	819.15	8894.45	4162.87	4020.19	69.89	7840.73	546.08	261.84	71.05	246.35

on average, about 15% of the flows are missing per sample across the test set. The missing values are then imputed using the mean values derived from the remaining available flows. As shown in Table II, our model achieves optimal performance on the Structured-Missing test set, outperforming all other baselines by a substantial margin. This result further validates the robustness of our model in handling such a structured failure scenario.

Although the Structured-Missing dataset exhibits a higher flow missing rate than RandomMissing-10%, our model maintains stable performance with only minor degradation. In contrast, PLNet and RouteNet suffer from significant performance drops, further validating the superiority of our model under traffic failure scenarios.

4) *Model Efficiency Analysis*: To verify the models' efficiency, we evaluate our model and baselines on a more common hardware configuration, with an Intel Xeon Gold 6130 CPU, 128 GB memory, and an NVIDIA GeForce RTX 3090 GPU with 24 GB VRAM. Table III summarizes the number of parameters, floating point operations (FLOPs), GPU memory usage, training time, and test time for each model. Although our model exhibits slightly higher complexity than most baselines in terms of parameters and FLOPs, this increase remains within reasonable bounds. Correspondingly, its GPU memory usage is well controlled, indicating that the increased complexity does not lead to excessive resource consumption. On the TnCwD dataset, our model requires 331 s per training epoch and 819.15 s for testing—significantly faster than FI-Graphormer, RouteNet, PLNet, and GAT, though moderately slower than NN, FCSGNN, DGCRN, STPGNN, and TimeMixer. Overall, as shown in Fig. 3, although our model incurs increased computational overhead, the resulting performance improvement is significant and justifies this cost.

C. Ablation Study

To further explore the effectiveness of each component in our model, three variant models were designed and compared against our model on the TnCwD dataset. These variants include

- 1) *w/o Random Sampling*: In this variant, the random sampling module is removed, and modeling is performed on the entire FIG.
- 2) *w/o Flow ID Embedding*: In this variant, flow node IDs are not embedded; instead, the node indices are used directly as ID information.

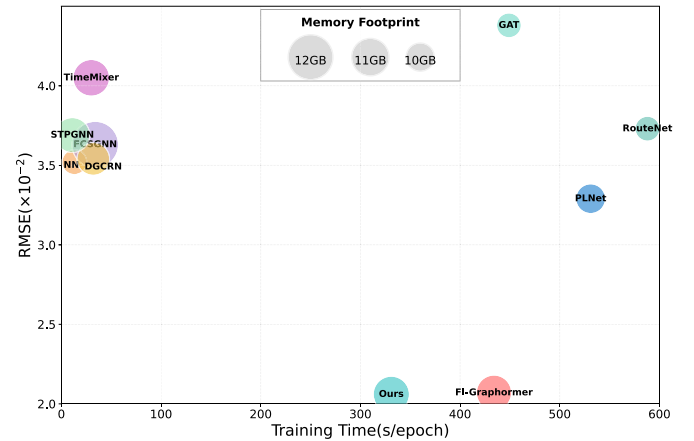


Fig. 3. Comparison of performance and complexity.

TABLE IV

COMPARISON RESULTS BETWEEN OUR MODEL AND ITS VARIANTS ($\times 10^{-2}$). THE BEST RESULTS ARE IN BOLD

Dataset	Metric	Model			
		Our model	w/o Random Sampling	w/o Flow Id Embedding	w/o Routing-Aware Attention
TnCwD	RMSE	2.06	2.36	2.21	2.70
	R ²	93.14	90.97	92.12	88.24
	PCC	96.62	95.65	96.38	94.30
	MAE	1.03	1.09	1.11	1.52

- 3) *w/o Routing-Aware Attention*: In this variant, the routing-aware attention module is removed. We directly concatenate the graph-level feature g with the initial flow features to form the FIG node features [i.e., replacing $\tilde{r}$ with g in (10)].

As shown in Table IV, the routing-aware attention module has the largest impact on model performance. Compared to our model, the w/o Routing-Aware Attention variant exhibits a decrease in R² and PCC by 5.26% and 2.40%, respectively. Meanwhile, RMSE and MAE increase by 31.07% and 47.57%, respectively. These results clearly illustrate that using attention to extract routing features most relevant to each flow significantly enhances model performance. In the w/o Random Sampling variant, the model exhibits a performance decline compared to our model. Generally, providing more complete information is beneficial to improve prediction performance. However, the result of this variant shows that learning from

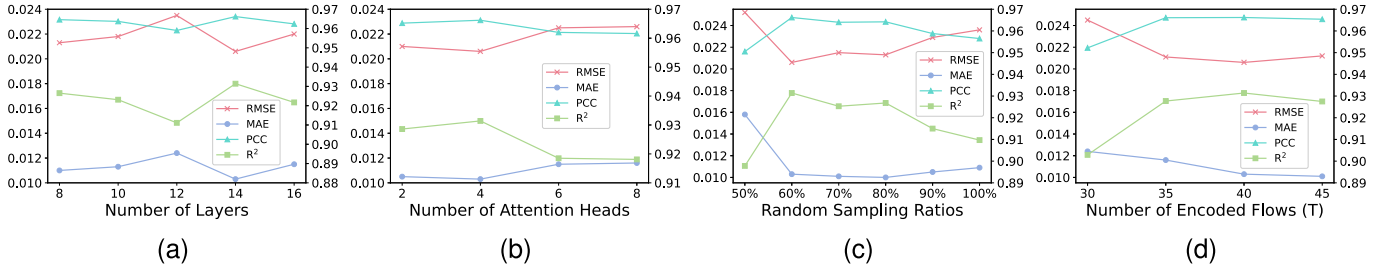


Fig. 4. Effect of various hyperparameter settings on the performance of our model. (a) Effect of the number of Graph Transformer encoder layers. (b) Effect of the number of attention heads in the Routing-Aware Attention and Graph Transformer modules. (c) Effect of the random sampling ratio for FIG subgraphs. (d) Effect of the number of encoded flows in line-graph node encoding.

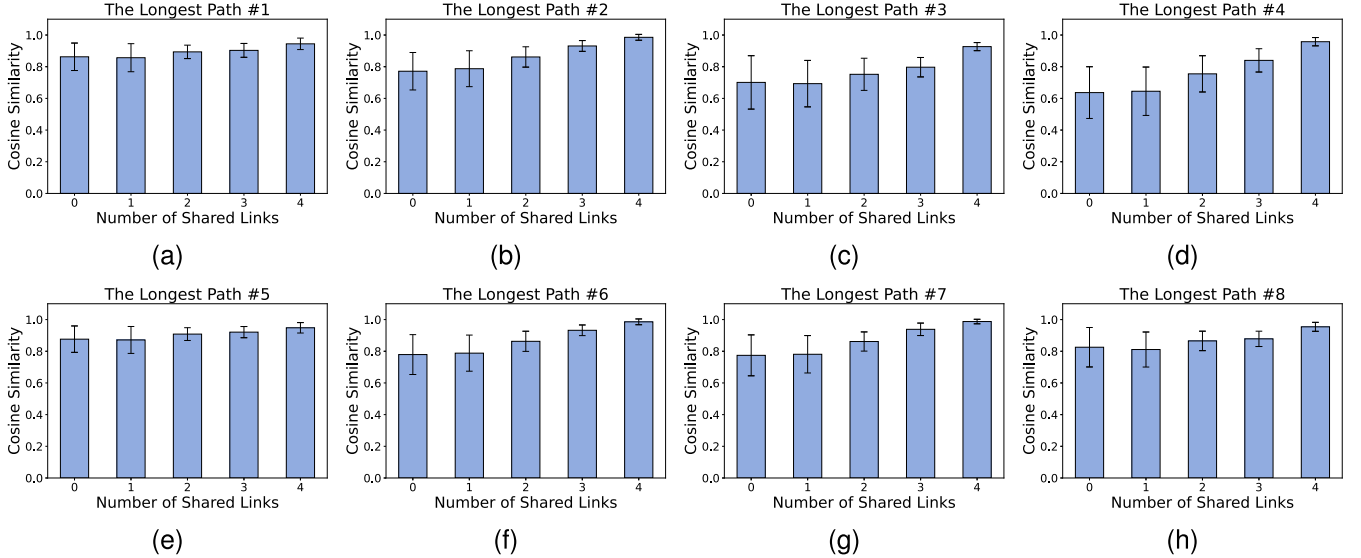


Fig. 5. Each plot shows the average cosine similarity between the routing-aware feature of a longest-path flow and groups of other flows that share 0-4 common links with that path. The increasing similarity trend demonstrates that the learned features encode routing similarity. (a)–(h) Present the cosine similarity analysis for Longest Paths #1 through #8, respectively.

masked samples can help the model better capture key patterns relevant to delay prediction. In the w/o Flow ID Embedding variant, the prediction performance also declines compared to our model. This indicates that incorporating learnable flow ID embeddings has a positive impact on delay prediction, as it helps the model better distinguish between different flows.

D. Hyperparameter Selection

In this section, we evaluate the sensitivity of our model to several critical hyperparameters, including the Transformer layer, the number of attention heads, the random sampling ratio and the fixed number of encoded flows. These experiments are conducted on the TnCwD dataset.

1) *Transformer Layer*: As shown in Fig. 4(a), increasing the number of layers does not consistently improve model performance. Instead, performance declines beyond a certain depth due to increased optimization difficulty and overfitting risk. When the number of layers reaches 14, the model achieves a good balance between capacity and training configuration, resulting in the best performance.

2) *Number of Attention Heads*: For simplicity, we set the number of attention heads in both the Routing-Aware Attention and the graph Transformer to the same value. As shown in Fig. 4(b), the model achieves optimal performance when the

number of attention heads is 4. Increasing the number of attention heads can improve model performance, but when the number exceeds 4, further increases lead to performance degradation. This indicates that too many attention heads do not bring performance gains and may instead introduce redundant noise or exacerbate overfitting.

3) *Random Sampling Ratios*: As shown in Fig. 4(c), random sampling ratios between 60% and 80% yield the best results. This suggests that overly low sampling ratios discard too much information, while excessively high ratios weaken the regularization effect and also underperform compared to the intermediate range. When the sampling ratio is set to 60%, most of the model's evaluation metrics reach their optimum; accordingly, we employ the sampling ratio of 60% in our experiments.

4) *Fixed Number of Encoded Flows*: To uniform the encoding length of line graph nodes, we employ a fixed number T of flows for encoding. In the TnCwD dataset, each link is traversed by multiple flows, with up to 40 flows pass through the same link. As shown in Fig. 4(d), the model achieves optimal performance when T is set to 40. When T is less than 40, the encoding suffers from an information bottleneck due to missing flow features, impairing the model's ability to capture flow-link relationships and thus degrading performance. When T exceeds 40, the encoding introduces redundant zero

TABLE V

COMPARISON OF THE MODEL GENERALIZATION PERFORMANCE ($\times 10^{-2}$). THE BEST RESULTS ARE IN BOLD. THE SECOND-BEST RESULTS ARE INDICATED WITH UNDERLINE

Test set	Metric	Model						
		Ours	w/o Routing-Aware Attention	w/o Random Sampling	FI-Graphormer	RouteNet	NN	PLNet
Substitution-A	RMSE	0.17	1.37	<u>0.23</u>	1.22	2.15	1.60	1.98
	R ²	99.94	96.77	<u>99.91</u>	97.40	92.02	95.59	93.21
	PCC	99.97	98.50	<u>99.96</u>	98.78	96.49	97.86	96.65
	MAE	0.12	0.62	<u>0.13</u>	0.31	1.07	0.90	1.27
Substitution-B	RMSE	1.98	2.40	<u>1.99</u>	2.12	2.52	2.50	2.75
	R ²	91.74	87.92	<u>91.71</u>	90.57	86.60	86.90	84.09
	PCC	96.91	95.21	<u>96.86</u>	96.65	94.14	95.14	93.05
	MAE	0.55	0.99	<u>0.56</u>	0.61	1.20	1.30	1.67
Unseen-A	RMSE	3.25	<u>3.08</u>	2.82	4.35	3.92	3.29	3.36
	R ²	89.11	86.79	<u>88.91</u>	73.56	78.53	84.85	84.20
	PCC	95.17	93.64	<u>95.06</u>	87.81	89.06	92.69	91.79
	MAE	1.15	1.56	<u>1.16</u>	2.73	2.74	1.91	2.01
Unseen-B	RMSE	4.18	4.36	4.39	5.66	5.09	4.65	<u>4.29</u>
	R ²	76.36	74.37	74.01	56.65	64.91	70.71	<u>75.10</u>
	PCC	89.58	87.71	<u>87.84</u>	78.77	80.67	86.37	86.86
	MAE	2.10	2.47	<u>2.28</u>	3.82	3.91	2.87	2.68

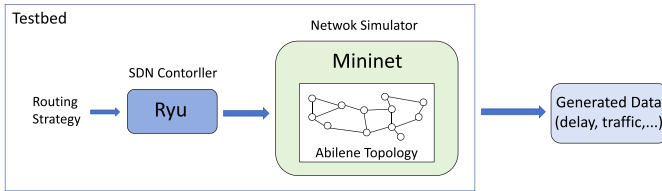


Fig. 6. Dataset used for routing generalization evaluation is generated using a testbed built with Mininet and Ryu.

padding, which distracts the model from meaningful patterns and increases the risk of overfitting.

E. Routing-Aware Feature Analysis

To further validate the effectiveness of the routing-aware attention mechanism, we demonstrate that routing information is, indeed, encoded in the output feature $\tilde{r}$. Empirically, the more similar two paths are in terms of routing, the more similar their corresponding feature representations tend to be. In the Abilene topology, the longest path consists of five hops, and there are eight such longest paths in total. For a specific longest path i , the remaining flows are grouped into five subsets based on the number of links they share with path i , ranging from 0 to 4. We then compute the cosine similarity between $\tilde{r}_i$ and the features of flows in each subset, and average the results within each group.

As shown in Fig. 5, the average cosine similarity increases with the number of shared links, indicating that the feature representations become more aligned as routing similarity increases. These results confirm that the proposed routing-aware attention mechanism effectively captures and encodes routing-related information.

F. Generalization for Dynamic Routing

To evaluate the routing generalization of our model, we adopt datasets originally generated by FI-Graphormer [21] using a Mininet and Ryu-based testbed on the Abilene topology. The datasets follow the same traffic and link parameter distributions as those used in the TnCwD dataset. Their generation process is illustrated in Fig. 6. Specifically, we utilize one training set and four testing sets, all carefully designed to evaluate routing generalization. To construct the training set, two distinct routing schemes are defined as follows.

- 1) *Routing Scheme A* adopts a shortest path first strategy with predefined configurations for all 132 OD flows on the Abilene topology. Notably, 21 of these flows have multiple equivalent shortest paths.
- 2) *Routing Scheme B* retains the same settings as Scheme A but forces the 21 flows to follow alternative shortest paths.

A total of 300 000 training samples are generated—200 000 under Scheme A and 100 000 under Scheme B. To assess the generalization performance, four testing sets are constructed

- 1) *Substitution-A* and *Substitution-B* are derived by partially modifying Routing Scheme A: the paths of 16 and 20 randomly selected OD flows, respectively, are replaced with their alternative shortest paths from Scheme B.
- 2) *Unseen-A* and *Unseen-B* introduce routing paths that are unseen during training. Starting from Scheme A, 10% and 20% of the OD flows, respectively, are reassigned to feasible routing paths not present in either Scheme A or Scheme B.

Using the training set, we train seven models and evaluate them on four testing sets. The results are summarized

in Table V. As observed, all models perform better on Substitution-A and Substitution-B than on Unseen-A and Unseen-B, indicating that the presence of previously unseen routing paths in the test data imposes a greater challenge and demands stronger generalization capabilities.

Across all four testing sets, our proposed model consistently achieves the best performance among all methods, demonstrating its superior generalization ability under dynamic routing scenarios. Notably, two ablated variants—w/o routing-aware attention and w/o random sampling—exhibit degraded performance compared with the full model, confirming the effectiveness of these two components in enhancing generalization. NN and PLNet also yield relatively competitive results on Unseen-A and Unseen-B, which can be largely attributed to their avoidance of sequential encoding. As a result, these models exhibit relatively robust performance when evaluated on testing sets containing previously unseen routing configurations.

V. CONCLUSION

In this article, we present a novel GNN-based model for end-to-end delay prediction that effectively addresses the generalization limitations of existing DL approaches. While prior models such as RouteNet and FI-Graphormer have achieved promising predictive performance, their reliance on sequential encoding severely hinders their ability to generalize to unseen routing schemes. To overcome this issue, we introduce a unified global routing representation that encodes both topological and routing information without the need for sequential processing. Combined with a routing-aware attention mechanism and a mask-based subgraph sampling strategy, our model is capable of retrieving routing-relevant features and learning global routing correlations from partial flow interactions. This design enhances both prediction accuracy and adaptability to dynamic routing scenarios. Extensive experiments on four public datasets—TnCwD, NSFNET, GBN, and GEANT2—demonstrate that our model not only outperforms state-of-the-art methods in accuracy but also exhibits strong generalization to routing configurations unseen during training. These results highlight the effectiveness of our proposed design and its potential for robust, intelligent network management in real-world environments. Future work will seek to enhance computational and memory efficiency to better support large-scale operational deployments and to further assess generalization performance under more diverse and dynamically changing routing environments.

REFERENCES

- [1] H. Taghizadeh, B. Safaei, A. Mahdi Hosseini Monazzah, E. Oustad, S. Rezagholi Lalani, and A. Ejlali, "LANTERN: Learning-based routing policy for reliable energy-harvesting IoT networks," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 6, pp. 6542–6554, Dec. 2024.
- [2] S. Rezagholi Lalani, B. Safaei, A. M. Hosseini Monazzah, H. Taghizadeh, J. Henkel, and A. Ejlali, "QUERA: Q-learning RPL routing mechanism to establish energy efficient and reliable communications in mobile IoT networks," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 4, pp. 1824–1839, Dec. 2024.
- [3] B. Sellami, A. Hakiri, S. B. Yahia, and P. Berthou, "Energy-aware task scheduling and offloading using deep reinforcement learning in SDN-enabled IoT network," *Comput. Netw.*, vol. 210, Jun. 2022, Art. no. 108957.
- [4] A. Sacco, M. Flocco, F. Esposito, and G. Marchetto, "Partially oblivious congestion control for the internet via reinforcement learning," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1644–1659, Jun. 2023.
- [5] L. Hui, M. Wang, L. Zhang, L. Lu, and Y. Cui, "Digital twin for networking: A data-driven performance modeling perspective," *IEEE Netw.*, vol. 37, no. 3, pp. 202–209, May 2023.
- [6] P. Almasan et al., "Network digital twin: Context, enabling technologies, and opportunities," *IEEE Commun. Mag.*, vol. 60, no. 11, pp. 22–27, Nov. 2022.
- [7] W. Jiang et al., "Graph neural networks for routing optimization: Challenges and opportunities," *Sustainability*, vol. 16, no. 21, p. 9239, Oct. 2024.
- [8] B. Kim, J. H. Kong, T. J. Moore, and F. T. Dagefu, "Deep reinforcement learning based routing for heterogeneous multi-hop wireless networks," 2025, *arXiv:2508.14884*.
- [9] E. Jafarnejad Ghomi, A. M. Rahmani, and N. N. Qader, "Applying queue theory for modeling of cloud computing: A systematic review," *Concurrency Comput., Pract. Exper.*, vol. 31, no. 17, p. 5186, Sep. 2019.
- [10] J. Vilaplana, F. Solsona, I. Teixidó, J. Mateo, F. Abella, and J. Rius, "A queuing theory model for cloud computing," *J. Supercomput.*, vol. 69, no. 1, pp. 492–507, Jul. 2014.
- [11] F. Ciucu and J. Schmitt, "Perspectives on network calculus: No free lunch, but still good value," in *Proc. Conf. Appl., Technol., Architectures, Protocols Comput. Commun.*, 2012, pp. 311–322.
- [12] F. Geyer and S. Bondorf, "DeepTMA: Predicting effective contention models for network calculus using graph neural networks," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1009–1017.
- [13] P. Cui, S. Han, X. Xu, J. Zhang, P. Zhang, and S. Ren, "End-to-end delay performance analysis of industrial Internet of Things: A stochastic network calculus perspective," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 5374–5387, Feb. 2024.
- [14] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and Tools for Network Simulation*, 2010, pp. 15–34.
- [15] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *Proc. 1st Int. ICST Conf. Simul. Tools Techn. Commun. Netw. Syst.*, 2008, pp. 1–10.
- [16] H. Wang, L. Wang, Q. Yu, Z. Zheng, A. Bouguettaya, and M. R. Lyu, "Online reliability prediction via motifs-based dynamic Bayesian networks for service-oriented systems," *IEEE Trans. Softw. Eng.*, vol. 43, no. 6, pp. 556–579, Jun. 2017.
- [17] M. Awad and R. Khanna, "Support vector regression," in *Efficient Learning Machines: Theories, Concepts, and Applications for Engineers and System Designers*. Berkeley, CA, USA: Apress, 2015, pp. 67–80.
- [18] A. Mestres, E. Alarcón, Y. Ji, and A. Cabellos-Aparicio, "Understanding the modeling of computer network delays using neural networks," in *Proc. Workshop Big Data Anal. Mach. Learn. Data Commun. Netw.*, Aug. 2018, pp. 46–52.
- [19] F. Krasniqi, J. Elias, J. Leguay, and A. E. C. Redondi, "End-to-end delay prediction based on traffic matrix sampling," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Jul. 2020, pp. 774–779.
- [20] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2260–2270, Oct. 2020.
- [21] W. Zheng et al., "End-to-end delay modeling via leveraging competitive interaction among network flows," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 2, pp. 1634–1647, Apr. 2024.
- [22] Q. Cao, M.-O. Pun, and Y. Chen, "Deep learning in network-level performance prediction using cross-layer information," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2364–2377, Jul. 2022.
- [23] Y. Chen, P. Yu, Z. Zheng, J. Shen, and M. Guo, "Modeling feature interactions for context-aware QoS prediction of IoT services," *Future Gener. Comput. Syst.*, vol. 137, pp. 173–185, Dec. 2022.
- [24] P. Zhang, W. Huang, Y. Chen, M. Zhou, and Y. Al-Turki, "A novel deep-learning-based QoS prediction model for service recommendation utilizing multi-stage multi-scale feature fusion with individual evaluations," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 2, pp. 1740–1753, Apr. 2024.
- [25] M. Wang et al., "Neural network meets DCN: Traffic-driven topology adaptation with deep learning," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 2, no. 2, pp. 1–25, 2018.
- [26] F. Geyer, "DeepComNet: Performance evaluation of network topologies using graph-based deep learning," *Perform. Eval.*, vol. 130, pp. 1–16, Apr. 2019.

- [27] Y. Li, R. Zemel, M. Brockschmidt, and D. Tarlow, "Gated graph sequence neural networks," in *Proc. ICLR*, 2016, pp. 1–19.
- [28] J. Li, P. Sun, and Y. Hu, "Traffic modeling and optimization in datacenters with graph neural network," *Comput. Netw.*, vol. 181, Nov. 2020, Art. no. 107528.
- [29] S. Xiao, D. He, and Z. Gong, "Deep-Q: Traffic-driven QoS inference using deep generative network," in *Proc. Workshop Netw. Meets AI ML-NetAI*, 2018, pp. 67–73.
- [30] Y. Yu, X. Si, C. Hu, and J. Zhang, "A review of recurrent neural networks: LSTM cells and network architectures," *Neural Comput.*, vol. 31, no. 7, pp. 1235–1270, Jul. 2019.
- [31] T. Suzuki, Y. Yasuda, R. Nakamura, and H. Ohsaki, "On estimating communication delays using graph convolutional networks with semi-supervised learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2020, pp. 481–486.
- [32] M. Ferriol-Galmés et al., "RouteNet-erlang: A graph neural network for network performance evaluation," in *Proc. IEEE Conf. Comput. Commun.*, May 2022, pp. 2018–2027.
- [33] M. Ferriol-Galmés et al., "RouteNet-Fermi: Network modeling with graph neural networks," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 3080–3095, Dec. 2023.
- [34] Y. Kong, D. Petrov, V. Räisänen, and A. Ilin, "Path-link graph neural network for IP network performance prediction," in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manage. (IM)*, May 2021, pp. 170–177.
- [35] S. Ding, J. Li, Y. Wu, D. Wang, and M. Zhang, "Transnet: A high-accuracy network delay prediction model via transformer and GNN in 6G," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2024, pp. 1–6.
- [36] M. Wang, L. Hui, Y. Cui, R. Liang, and Z. Liu, "XNet: Improving expressiveness and granularity for network modeling with graph neural networks," in *Proc. IEEE Conf. Comput. Commun.*, May 2022, pp. 2028–2037.
- [37] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [38] X. Hei, J. Zhang, B. Bensaou, and C.-C. Cheung, "Wavelength converter placement in least-load-routing-based optical networks using genetic algorithms," *J. Opt. Netw.*, vol. 3, no. 5, pp. 363–378, 2004.
- [39] J. Pedro, J. Santos, and J. Pires, "Performance evaluation of integrated OTN/DWDM networks with single-stage multiplexing of optical channel data units," in *Proc. 13th Int. Conf. Transparent Opt. Netw.*, Jun. 2011, pp. 1–4.
- [40] F. Barreto, E. C. G. Wille, and L. Nacamura, "Fast emergency paths schema to overcome transient link failures in ospf routing," 2012, *arXiv:1204.2465*.
- [41] H. Taud and J. Mas, "Multilayer perceptron (MLP)," in *Geomatic Approaches for Modeling Land Change Scenarios*. Cham, Switzerland: Springer, 2017, pp. 451–455.
- [42] P. Velickovic et al., "Graph attention networks," *Stat.*, vol. 1050, no. 20, pp. 10–48550, 2017.
- [43] Y. Wang et al., "Fully-connected spatial-temporal graph for multivariate time-series data," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 14, pp. 15715–15724.
- [44] F. Li et al., "Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution," *ACM Trans. Knowl. Discovery Data*, vol. 17, no. 1, pp. 1–21, Feb. 2023.
- [45] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [46] W. Kong, Z. Guo, and Y. Liu, "Spatio-temporal pivotal graph neural networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 8, pp. 8627–8635.
- [47] S. Wang et al., "TimeMixer: Decomposable multiscale mixing for time series forecasting," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024, pp. 38626–38652.



Yiqi Chen received the B.E. degree from South China Normal University, Guangzhou, China, in 2022, where he is currently pursuing the master's degree with the School of Computer Science. His research interest focuses on time-series modeling.



Dongwei Liu received the B.E. degree from South China Normal University, Guangzhou, China, in 2024, where he is currently pursuing the master's degree with the School of Computer Science. His research interests include time-series modeling.



Huile Wang received the B.E. degree from Nanjing Agricultural University, Nanjing, China, in 2022. He is currently pursuing the master's degree with the School of Computer Science, South China Normal University, Guangzhou, China. His research interests include time-series analysis and large language model optimization for time-series forecasting.



Yusong Zhou received the B.E. degree from the University of Shanghai for Science and Technology, Shanghai, China, in 2022. He is currently pursuing the master's degree with the School of Computer Science, South China Normal University, Guangzhou, China. His research interests focus on time-series modeling.



Zichen Wang received the B.E. degree from Three Gorges University, Yichang, China, in 2023. She is currently pursuing the master's degree with the School of Computer Science, South China Normal University, Guangzhou, China. Her research interests include network modeling and network optimization.



Weiping Zheng received the B.E. degree from Shantou University, Shantou, China, in 2000, the M.S. degree from South China Normal University, Guangzhou, China, in 2003, and the Ph.D. degree from South China University of Technology, Guangzhou, in 2010. He is currently a Professor with the School of Computer Science, South China Normal University. His research interests include time-series modeling and cloud computing.